

PRODUCTION SYSTEM DOCUMENTATION • VERSION 2.1

Forsa Financial News Monitoring Agent

How the Autonomous, Headless AI Intelligence System Gathers, Filters, Deduplicates, Verifies, and Delivers Executive Briefings for Consumer Finance Leadership

Organization: Forsa (Drive Finance), Cairo, Egypt **Regulatory Domain:** FRA Law No. 18 of 2020 **Engine:** LangGraph State Machine

Active Schedule: Daily Execution (Morning Cadence) **Configurable Via GitHub Actions:** Weekly Consolidated Digest

1 Executive Summary & Core Purpose

The **Forsa Financial News Monitoring Agent** is an autonomous, production-grade artificial intelligence system engineered specifically for the Chief Executive Officer and Executive Committee of **Forsa** (Drive Finance), one of Egypt's premier consumer finance and Buy-Now-Pay-Later (BNPL) institutions.

The agent actively runs on a **daily automation schedule**, monitoring Egyptian consumer lending news, competitor moves, and Financial Regulatory Authority (FRA) announcements as they unfold every morning. In addition, via **GitHub Actions**, the system can be configured or dispatched to generate a **weekly consolidated digest**, aggregating and synthesizing the entire 7-day picture for Sunday executive committee meetings.

⚠ The Information Bottleneck

- Information Overload:** Hundreds of irrelevant daily articles on central bank interest rates, commercial bank deposits, and commodity prices.
- Syndicated Repetition:** A single corporate press release syndicated across 10+ Egyptian portals with minor headline variations.
- Transliteration Inconsistencies:** Brand names distorted in translation (e.g., *Souhoola* as *SooLa*, *valU* as *Valiu*).
- AI Hallucination Risk:** Standard chat LLMs inventing speculative opinions rather than grounded regulatory facts.

✅ The Agent's Solution

- Dual Cadence Flexibility:** Runs daily for active morning intelligence; configurable weekly via GitHub Actions.
- 5-Tier Deduplication:** Groups syndicated media coverage into unified canonical events.
- Deterministic Brand Dictionary:** Normalizes Egyptian fintech names to canonical corporate spellings.
- Strict Factuality:** Grounded summaries containing strictly numbers, dates, decisions, and source links.

💡 Architectural Principle: Why This System is NOT a Chatbot

The Forsa News Agent is completely **headless, non-conversational, and autonomous**. It does not wait for user prompts. Built on **LangGraph**, it is an autonomous state machine that triggers automatically on its scheduled run (daily morning flow or weekly GitHub Action dispatch), executes an end-to-end 9-stage data and AI pipeline, updates a persistent PostgreSQL database ledger to guarantee 100% idempotency, delivers a curated intelligence digest to the executive inbox, and cleanly terminates.

Document Table of Contents

Chapter 1: Executive Summary & Core Purpose	Page 1
Chapter 2: Domain Scope & Dual Scheduling (Daily Production vs. Weekly GitHub Actions)	Page 2
Chapter 3: The End-to-End Pipeline: Ingestion & Preprocessing (Stages 1–2)	Page 3
Chapter 4: Classification Funnel & 5-Tier Deduplication (Stages 3–4)	Page 4
Chapter 5: Source Verification, Idempotency & Summarization (Stages 5–7)	Page 5
Chapter 6: Executive Email Delivery & "No-News" Protocol (Stages 8–9)	Page 6
Chapter 7: Cloud Orchestration, Dual-Cadence Scheduling & Database Architecture	Page 7
Chapter 8: Production Reliability, Safety Defenses & Lifecycle Cheat Sheet	Page 8

2 Domain Scope & Dual-Cadence Scheduling

A primary differentiator of the Forsa News Agent is its strict domain filtering. Standard financial aggregators fail because they indiscriminately forward macroeconomic commentary, commercial bank deposit updates, and commodity price trends. The Forsa agent is calibrated strictly for an Egyptian consumer lending and BNPL institution.

🏛️ Strategic Distinction: Why Central Bank of Egypt (CBE) News is Excluded

Forsa is a Non-Banking Financial Institution (NBFI) licensed and regulated by the **Financial Regulatory Authority (FRA)** pursuant to Egyptian Consumer Finance Law No. 18 of 2020. Forsa is *not* a commercial bank regulated by the Central Bank of Egypt (CBE). Consequently, CBE Monetary Policy Committee rate decisions, commercial banking reserve requirements, and interbank liquidity operations are intentionally filtered out. This keeps executive briefings strictly focused on consumer lending, BNPL installment regulations, credit caps, and direct market competitors.

Monitoring Scope Boundary Matrix

In-Scope Intelligence (Retained & Prioritized)	Out-of-Scope Noise (Filtered & Dropped)
REGULATORY Financial Regulatory Authority (FRA): Circulars, supervisory decrees, board decisions, and licensing guidelines.	DROPPED Central Bank of Egypt (CBE): Commercial banking interest rates, MPC policy statements, and reserve ratios.
LENDING Consumer Finance Dynamics: Law 18/2020 updates, consumer debt burden limits, BNPL frameworks, and retail credit regulations.	DROPPED Sovereign Debt & Treasuries: Egyptian government Treasury Bill (T-bill) and Treasury Bond (T-bond) auction results.
COMPETITORS Competitor Intelligence: Product launches, merchant partnerships, expansions, and debt issuances by valU, MNT-Halan, Souhoola, Contact, Aman, Sympl, Blnk, Fawry, Shahry.	DROPPED Commodities & Shipping: Food prices, poultry, wheat tenders, fertilizer, Suez Canal freight rates, and oil indices.
INFRASTRUCTURE Credit & Identity: I-Score bureau regulations, e-KYC digital onboarding, OTP verification rules, and digital signature compliance.	DROPPED Academic & Ceremonial: University hackathons, "Got Talent" contests, student competitions, and internal company awards.
CAPITAL Securitization & Markets: Securitized bond tranches by consumer finance companies and weekly EGX30 benchmark recap.	DROPPED Traditional Insurance: Marine, cargo, fire, and agricultural insurance syndication pools unrelated to retail lending.
FINTECH Payment Infrastructure: Digital wallets, payment aggregators, and national payment rails (InstaPay) affecting consumer transactions.	DROPPED Clickbait & Stubs: Rumor pieces lacking dates, concrete financial figures, or official verification.

Dual Scheduling Architecture: Daily vs. Weekly Execution

🔥 Active Production: Daily Morning Schedule

The automation schedule is actively working on a **daily cadence**. Operating with a 24-hour lookback window, it delivers morning intelligence to leadership so that breaking competitor merchant deals, consumer app updates, and urgent FRA circulars are reviewed within hours.

Active Production Default

🔧 Configurable Via GitHub Actions: Weekly Digest

Via **GitHub Actions** (`weekly_news.yml`), the schedule can be configured or triggered to execute **weekly** with a 7-day lookback (`lookback_days: '7'`). This consolidates the entire week's developments into a comprehensive strategic executive digest.

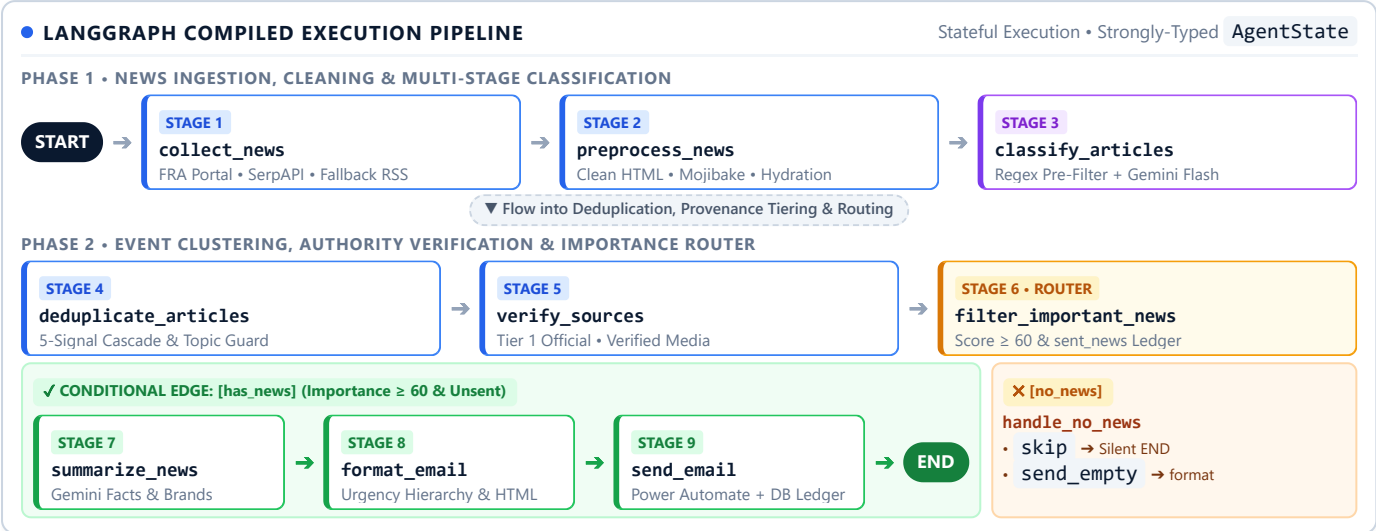
GitHub Action Workflow Dispatch

🔒 Absolute Idempotency & Safety Overlap Guarantee

Whether running daily or weekly, the ingestion window automatically incorporates a **6-hour safety overlap** (`OVERLAP_HOURS = 6`) to capture stories published around execution boundaries. The persistent PostgreSQL `sent_news` ledger table guarantees that events delivered in previous runs are never emailed again.

3 The 9-Stage Pipeline: Ingestion & Preprocessing

The core pipeline is implemented as a compiled state graph using **LangGraph**. Execution moves deterministically through 9 specialized functional nodes, accumulating verified metadata in a strongly-typed state object (`AgentState`).



4

Classification Funnel & Deduplication

Raw financial news volume is substantial. Sending every ingested article to an LLM is both slow and expensive. The agent employs an efficient 3-stage filtration funnel followed by a 5-tier deduplication cascade.

STAGE 3

Multi-Stage Classification Funnel (Rule + LLM)

app/graph/nodes/classify.py

Combines zero-cost deterministic rules with Google Gemini Flash to classify and score candidates:

- Stage 1 — Deterministic Keyword Pre-Filter (Zero Token Cost):** Evaluates articles against compiled regex patterns. Articles containing zero consumer finance/regulatory keywords, or matching out-of-scope triggers (e.g., CBE rates, student contests, grain prices), are rejected instantly at zero token cost.
- Stage 2 — Deterministic Domain Rules:** Flags high-confidence matches (e.g., direct FRA portal items automatically receive 100 relevance and official category assignment).
- Stage 3 — Google Gemini Flash Classification:** Evaluates remaining candidate articles, assigning category (*FRA*, *Consumer Finance*, *Competitor*, *FinTech*), subcategory, detected entities, competitor matches, a relevance score (0–100), and an importance score (0–100).

Inputs:

clean_articles

Outputs:

relevant_articles

(Score >= 50)

Efficiency:

Drops ~70% of noise prior to LLM

STAGE 4

5-Tier Multi-Signal Deduplication Stack

app/graph/nodes/deduplicate.py

In Egypt, a single corporate partnership or bond issuance is often re-published by 10+ financial outlets with slightly altered headlines. The agent collapses these into a single unified `NewsEvent` using an ordered 5-signal cascade:

Signal Layer	Matching Mechanism	Threshold / Behavior
1. Canonical URL	Normalized URL matching after stripping UTM parameters and tracking tokens.	Exact match; instant collapse.
2. Content SHA-256	Cryptographic hash of cleaned body text.	Identical syndicated press release bodies.
3. Fuzzy Title Match	Levenshtein Token Sort Ratio comparing key terms across headlines.	Token Sort Ratio >= 85%.
4. Entity & Time Window	Shared named entities (e.g., "valU" + "Amazon") within an active time window.	Domain-specific entity overlap.
5. Vector Embeddings	Cosine similarity of Google Gemini text embeddings generated on title + first 500 chars.	Cosine Similarity >= 0.85.

Cross-Topic Protection Guard:

Ensures that distinct official decisions issued on the same day (e.g., FRA I-Score Circular No. 174 vs. Real Estate Valuation Decree No. 191) are recognized as separate events and never improperly merged.

Inputs:

relevant_articles

Outputs:

Unified NewsEvent clusters

Cluster Authority:

Selects most official source as canonical

5

Verification, Idempotency & Summarization

Before news reaches executive leadership, it must be verified for institutional credibility, filtered for high strategic impact, checked against past deliveries, and synthesized with zero AI hallucination.

STAGE 5

Source Verification & Provenance Tiering

app/graph/nodes/verify.py

Leadership must know whether an item is an official decree, verified financial press, or uncorroborated media:

Tier 1: OFFICIAL

FRA portal, Egyptian Gazette, government gazette decrees.

Tier 2: VERIFIED

Enterprise, Al Borsa, Daily News Egypt, Amwal Al Ghad, Economy Plus.

Tier 3: UNVERIFIED

General blogs, aggregators; requires corroboration.

Inputs: events

Outputs: Events with verification badges

Audit: Source authority preserved

STAGE 6

Importance Filtering & Idempotency Ledger

app/graph/nodes/filter.py

Enforces executive selectivity and guarantees that the CEO is never sent the same intelligence twice:

- Importance Cutoff:** Events are evaluated on a 0–100 scale using Gemini Flash. Only events scoring ≥ 60 (the `IMPORTANCE_THRESHOLD`) are approved for executive delivery.
- Database Idempotency Check:** Queries the PostgreSQL `sent_news` ledger table. If an event or canonical story was already sent to leadership in any previous run (whether yesterday or weeks ago), it is filtered out.
- Conditional Routing:**
 - If important, unread events exist → routes forward to `summarize_news`.
 - If zero events pass the threshold → routes to `handle_no_news`.

Inputs: events

Outputs: important_events

Ledger: PostgreSQL historical verification

STAGE 7

Factual Summarization & Brand Normalization

app/graph/nodes/summarize.py

Synthesizes each approved event into an executive briefing using Google Gemini under strict prompt constraints:

- Zero Speculation / No AI Advice:** Strictly prohibited from generating opinions, editorializing, or answering "What this means for Forsa." Produces only verifiable facts, figures, dates, and decrees.
- Stub Rejection:** Discards articles that lack substance, concrete dates, or institutional decisions.
- Deterministic Brand Dictionary (`spelling.py`):** Standardizes phonetic Arabic transliterations into canonical corporate spellings:

Raw Transliteration / Arabic	Canonical Executive Output
SooLa, Sohoola, Sahula, سهولة	Souhoola
Valiu, Falio, Valyou, فاليو	valU
Mnt-Halan, MNT Halan, حالا	MNT-Halan
Blink Consumer Finance, بلك	Blnk
Simple BNPL, Sembl, سيمبل	Sympl
Faury, Fawri, فوري	Fawry
i-score, iscore, IScore, اي سكور	I-Score

Inputs: important_events

Outputs: Attached EventSummary

Quality: Rejects stubs and speculative text

6 Executive Email Delivery & Formatting

The final delivery stage formats the curated intelligence into an executive briefing and delivers it through enterprise communication channels with full operational auditing.

STAGE 8Executive Digest Layout & "No-News" Protocolapp/graph/nodes/format_email.py

Assembles a responsive HTML and plain-text briefing designed for Microsoft Outlook and mobile mail clients:

- Visual Category Urgency Hierarchy:**
 - FRA & Regulation:** Placed at the top (compliance mandates, licensing).
 - Consumer Finance & Competitors:** Competitor product offers, merchant deals, securitization.
 - FinTech & Financial Market:** Digital payment infrastructure, wallets, EGX30 recap.
- Information Hygiene:** Internal system metrics (such as relevance scores, importance numbers, or AI prompts) are strictly stripped from the email. Executives see only the headline, structured summary, verified source, publication date (Cairo time), and direct article link.
- Configurable "No-News" Handling:**
 - `NO_NEWS_BEHAVIOUR=skip` (Default): Silent termination. Zero emails sent, preventing executive inbox clutter on quiet days.
 - `NO_NEWS_BEHAVIOUR=send_empty` : Sends a brief confirmation confirming the agent ran successfully and found no material issues.

Inputs: Curated summaries Outputs: email_html , email_plain Timezone: Standardized to Africa/Cairo

STAGE 9Enterprise Delivery & Run Telemetry Persistenceapp/graph/nodes/send_email.py

Dispatches the intelligence briefing and records full audit records in the database:

1. Power Automate

Dispatches JSON payload to webhook.
Uses native Outlook connector. **Zero IT admin friction.**

2. M365 Graph API

Direct OAuth 2.0 client credentials via Microsoft Graph API (`Mail.Send`).

3. Standard SMTP

Native Python `smtplib` with STARTTLS or SSL encryption.

Ledgering & Telemetry: Records each delivered event into the `sent_news` table and creates a comprehensive record in `workflow_runs` (tracking articles collected, deduplicated, verified, and delivered).

Inputs: Formatted email Outputs: Delivery status & run statistics Audit: Complete execution telemetry logged

Executive Digest Format Preview

Executive emails are styled with corporate branding, clean borders, and clear metadata badges. Each item includes: **Canonical Headline**, **Factual Summary** (2–3 bullet points), **Verified Source Badge**, **Publication Date** (Cairo time), and a **Direct Source Link**.

7 Cloud Orchestration & Database Architecture

Dual-Cadence Scheduling: Daily Production vs. Weekly GitHub Actions

The architecture decouples the execution engine from the schedule, enabling both daily and weekly intelligence cadences:

 **Daily Production Automation Schedule**

The active production schedule triggers **daily** via Microsoft Power Automate Scheduled Cloud Flow (Recurrence: **1 Day**). It spins up the runner, collects the past 24 hours of Egyptian financial news, and delivers a concise morning briefing directly to executive inboxes.

Currently Active Schedule

 **Weekly Consolidation Via GitHub Actions**

Via **GitHub Actions** (`weekly_news.yml`), the schedule can be configured or triggered to run **weekly** by passing `lookback_days: '7'` . This aggregates all regulatory and competitor events across the full 7-day week into a single executive digest.

Configurable On-Demand / Scheduled

- How to Configure Weekly Mode in GitHub Actions:**
- **Via GitHub UI:** Actions → *Financial News Digest* → *Run workflow* → Set `Lookback Days` to `7` .
 - **Via Power Automate Recurrence:** Change Recurrence Frequency from `Day` to `Week` (Sunday 08:30 AM Cairo) and set JSON body: `{"inputs": {"lookback_days": "7"}}` .
 - **Via GitHub CLI:** `gh workflow run weekly_news.yml -f lookback_days=7` .

Relational Persistence Model (PostgreSQL & SQLAlchemy Async)

The agent persists state, historical intelligence, and audit trails across 7 relational tables managed by SQLAlchemy 2.0 Async (`asyncpg`) and Alembic:

Database Table	Stored Content & Entity Relationships	Operational Purpose
<code>sources</code>	News portals, FRA official portal, RSS feed URLs, language, and authority tier (1, 2, or 3).	Source registry & provenance verification.
<code>competitors</code>	Monitored Egyptian fintechs (valU, MNT-Halan, Souhoola, Blnk, etc.) with alias spellings.	Entity matching & brand dictionary seeds.
<code>articles</code>	Ingested raw articles: title, canonical URL, SHA-256 hash, raw text, and published timestamp.	Ingestion history & full-text cache.
<code>article_classifications</code>	Category, subcategory, entities JSON, relevance score, importance score, and confidence.	Machine learning & LLM audit log.
<code>events</code>	Unified deduplicated events: canonical title, summary, event date, verification status.	Curated intelligence events.
<code>sent_news</code>	Foreign key links to sent <code>events</code> , recipient email address, run ID, and timestamp.	Idempotency ledger (prevents duplicate emails).
<code>workflow_runs</code>	Execution records: start/completion timestamps, status, counts of collected/sent items, error logs.	Operational health & executive reporting.

8 Production Reliability & Cheat Sheet

Rate Limiting & API Pacing

To prevent Google Gemini API 429 quota exhaustion during high-volume periods, all LLM calls use sequential pacing with `asyncio.Semaphore(1)` and 2–4 second sleep intervals.

Pre-Flight Search Circuit Breaker

Before executing expensive search API calls, `serpapi_checker.py` queries account quotas. If quota is depleted (< 5 searches), it flips immediately to the free RSS/scrapper fallback without failing.

Isolated Collector Failures

Every web scraper and RSS collector executes inside an isolated `asyncio.gather(..., return_exceptions=True)` block. If one news portal is down or times out, the other sources continue unaffected.

Atomic State Persistence

All database operations use atomic nested transactions (`session.begin_nested()`). If a single article insertion encounters a duplicate key error, only that row is rolled back.

Pipeline Lifecycle Quick-Reference Cheat Sheet

Step	Node Name	Primary Action	Core Defense / Quality Guarantee
1	collect_news	Gathers from FRA, SerpAPI, and RSS fallbacks.	Automatic fallback prevents API quota blockage.
2	preprocess_news	Sanitizes HTML, fixes mojibake, hydrates full text.	Ensures LLM receives clean, complete Unicode text.
3	classify_articles	Filters out-of-scope news and scores relevance.	Drops CBE/commodity noise at zero token cost.
4	deduplicate_articles	Groups syndicated stories into single events.	5-signal cascade eliminates media redundancy.
5	verify_sources	Assigns OFFICIAL, VERIFIED, or UNVERIFIED tiers.	Enforces clear regulatory vs. media provenance.
6	filter_important_news	Applies 60-point cutoff & checks <code>sent_news</code> ledger.	Guarantees 100% idempotency (zero repeat emails).
7	summarize_news	Generates factual summaries and normalizes brands.	Anti-hallucination constraint & brand dictionary.
8	format_email	Builds mobile-responsive executive digest.	Strips internal scores; categorizes by urgency.
9	send_email	Dispatches via Power Automate / Graph / SMTP.	Zero IT-admin friction; records run telemetry.

Operational Commands Reference (Daily & Weekly Runs)

```
# Active Production: Run daily execution (1-day lookback)
py -3 app/main.py --run-now --days 1

# Weekly Mode: Run consolidated 7-day lookback execution
py -3 app/main.py --run-now --days 7

# Trigger weekly consolidation run via GitHub CLI
gh workflow run weekly_news.yml -f lookback_days=7

# Start continuous scheduler daemon in background
py -3 app/main.py
```